

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería Electrónica

Verificación Funcional – Proyecto 1 (25 %)

Ambiente de Verificación en Capas para Bus con Árbitro

Documentación técnica del ambiente de verificación

Integrantes

Gerald Blanco Sáenz
Leonardo Pérez Sandoval

Fecha de entrega: Jueves 1 de octubre de 2026

Repositorio del proyecto: <https://github.com/GeraldBS/Proyecto-1-Verificaci-n-funcional>

Índice

1. Objetivo	2
2. Descripción del DUT	2
2.1. Parámetros del DUT	2
2.2. Arquitectura interna del DUT	3
2.3. Formato del paquete a nivel del DUT	3
3. Arquitectura del ambiente de verificación	3
3.1. Función de cada componente	4
3.2. Interfaz entre el ambiente y el DUT	5
4. Transacciones y estructuras de datos del ambiente	6
5. Ejecución concurrente del ambiente	6
6. Generador y aleatorización	7
6.1. Número de transacciones por terminal	7
6.2. Largo del paquete (16, 32, 64)	7
6.3. Tiempos de envío / retardo entre mensajes	7
6.4. Mensajes con error (destinos inexistentes)	8
6.5. Direcciones de destino y origen	8
6.6. Identificador de <i>broadcast</i>	8
6.7. Generador: reparto por turnos	8
7. Driver y Monitor	9
8. Scoreboard y Checker	12
9. Casos de esquina	14
10. Generación de resultados: reporte CSV e histograma	16
10.1. Resultados de una corrida real y verificación	17
10.2. Cobertura de los tres anchos de paquete (16, 32 y 64 bits)	19
10.3. Semilla de aleatorización: fija vs. variable	19
10.4. Matriz completa: los tres anchos cruzados con tres tamaños de bus	19
10.5. Barrido dirigido de distribución y temporización de tráfico	21
10.6. Hallazgo: la coincidencia <i>push/pop</i> depende del ancho del paquete	22
11. Cómo compilar y correr el ambiente	22
11.1. Pruebas unitarias dirigidas previas al ambiente completo	22
12. Conclusión	23
A. Anexo: código fuente completo del ambiente	23
B. Anexo: pruebas unitarias dirigidas	24

1 Objetivo

Definir y documentar la estrategia de verificación funcional del DUT (un bus compartido con árbitro y FIFOs por dispositivo), especificando la arquitectura del ambiente de pruebas, el formato de los paquetes de comunicación entre los bloques del ambiente, y las funcionalidades del diseño que son cubiertas mediante pruebas con aleatorización controlada (*constrained-random*), incluyendo los casos de esquina relevantes.

2 Descripción del DUT

El DUT es un bus compartido al que se conectan M dispositivos, cada uno con su capa de protocolo. El DUT *no* incluye las FIFOs de entrada y salida: expone por cada dispositivo las señales `pndng/pop` y `push/D_push` esperando que alguien se las provea, y es el ambiente de verificación el que las emula (§7). El acceso al bus es gestionado mediante un esquema de arbitraje distribuido: cada interfaz de dispositivo administra esas señales y participa en el intercambio de un “turno” que habilita, de forma única en cada instante, el paso de datos entre los serializadores (SRLZ) de entrada y salida de un solo dispositivo hacia el bus.

2.1 Parámetros del DUT

Parámetro	Valor por defecto	Descripción
<code>bits</code>	1	Número de buses independientes instanciados (el enunciado del proyecto lo deja en 1; no se modifica).
<code>drvrs</code>	4	Número de dispositivos (<i>drivers</i>) conectados al bus. Modificable; máximo práctico 254 para poder generar destinos inexistentes con direcciones de 8 bits.
<code>pckg_sz</code>	16	Ancho en bits del paquete de datos. Debe soportar 16, 32 y 64 (parámetro estructural, fijado en tiempo de compilación).
<code>broadcast</code>	<code>{8{1'b1}}</code>	Identificador reservado de dirección de destino (0xFF) que representa un mensaje de difusión hacia todos los dispositivos. No se modifica: el DUT compara internamente contra este valor fijo de 8 bits.

Tabla 1: Interfaz del DUT, según el prototipo `bs_gnrtr_n_rbtr`.

Estos valores se fijan al compilar mediante `+define+PCKG_SZ` y `+define+DRVRS` y se centralizan en el ambiente de pruebas en el archivo `tb/parametros.svh`.

2.2 Arquitectura interna del DUT

Por cada dispositivo $i \in [0, \text{drvrs} - 1]$ el DUT instancia una interfaz `bs_ntrfs_n_rbtr`, que combina:

- **Control y arbitraje distribuido** (`ntrfs_cntrl_n_rbtr`): un contador local (`Counter_arb`) avanza con cada pulso de la señal compartida `trn_chng` (el “testigo” que pasa de un dispositivo a otro). Cuando el conteo coincide con el identificador propio (`ntrfs_id`) y el dispositivo tiene una solicitud pendiente (`bs_rqst`), obtiene el permiso de transmisión (`bs_grnt`) y controla temporalmente las líneas compartidas `bus` y `bs_bsy` mediante buffers triestado.
- **Máquinas de escritura y lectura** (`Write_st_Mchn`, `Read_st_Mchn`): controlan cuándo el dispositivo con el permiso serializa su dato saliente hacia `bus`, y cuándo cada dispositivo (incluido el emisor) deserializa lo que circula por el bus hacia su propio registro `D_push`.
- **Serializadores** (`serializer`): el paquete de `pckg_sz` bits se transmite *bit a bit* por una única línea compartida `bus`; cada dispositivo compara, mientras el dato entra, los bits altos contra su propio identificador o contra el valor de `broadcast` para decidir si debe seguir capturando el resto del paquete.

Este esquema logra que el bus se comporte, desde el punto de vista del ambiente de verificación, como un medio compartido de acceso exclusivo con turnos rotativos (*round-robin*) entre los `drvrs` dispositivos, sin exponer un árbitro central como bloque aparte.

Una particularidad de este esquema condiciona al ambiente: el contador `Counter_arb` tiene reset asíncrono por *flanco* de subida, y su reloj es la señal compartida `trn_chng`, que arranca en `X`. Si `reset` se mantiene en 1 desde el tiempo cero nunca se produce ese flanco, el contador queda indefinido y el bus no llega a funcionar. Por eso `agent_drv::reset_dut()` aplica el reset como un pulso $0 \rightarrow 1 \rightarrow 0$ y no como un nivel inicial.

2.3 Formato del paquete a nivel del DUT

A nivel de las señales `D_push/D_pop` del RTL, el paquete es una palabra de `pckg_sz` bits sin estructura explícita para el DUT: son los 8 bits más altos los que el DUT interpreta como dirección de destino (o `broadcast`), y el resto como carga útil. Esta convención es la que el ambiente de verificación debe respetar al construir cada transacción.

3 Arquitectura del ambiente de verificación

El ambiente se implementa como un *testbench* en capas (*signal*, *command*, *functional*, *scenario* y *test*), siguiendo los lineamientos generales del curso. Por indicación del profesor, para este proyecto no se implementan las capas de *Assertions* ni de *Functional Coverage*; la verificación de correctitud recae completamente en el **Scoreboard** y el **Checker**.

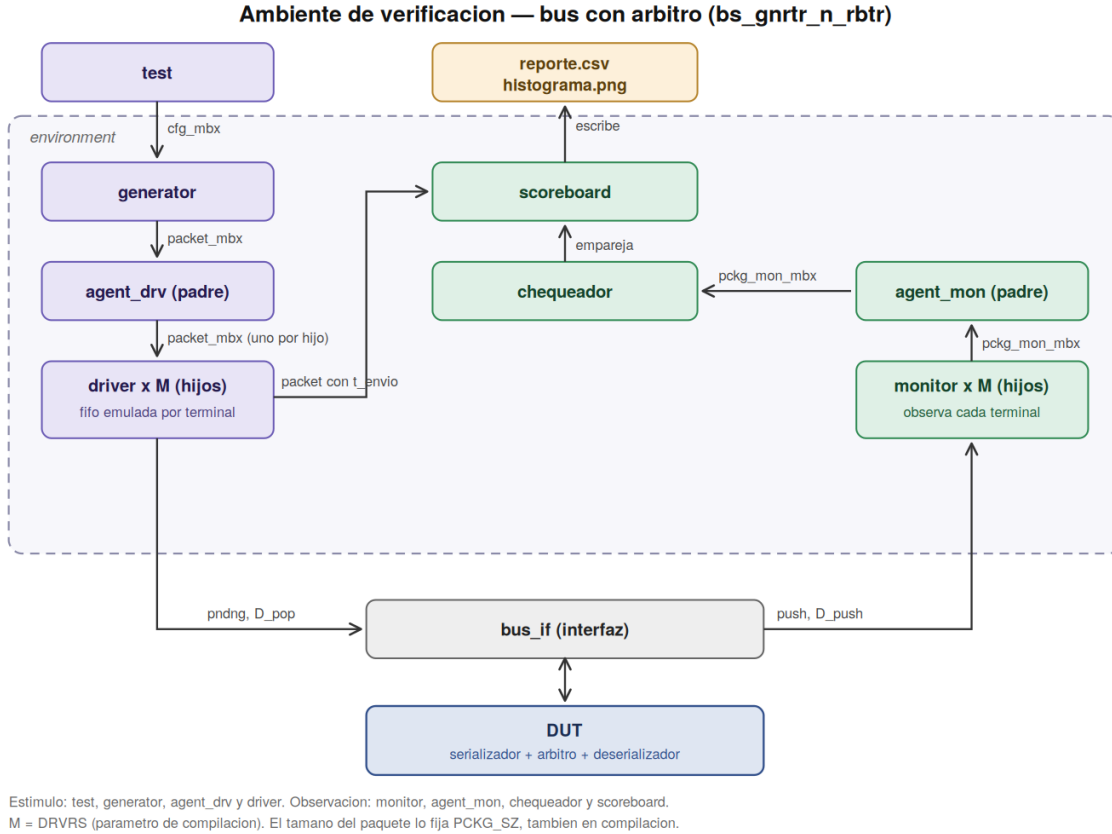


Figura 1: Diagrama completo del ambiente de verificación: bloques, mailboxes de comunicación entre ellos y su conexión con el DUT a través de la interfaz `bus_if`.

3.1 Función de cada componente

Componente	Función
<code>test</code>	Define las restricciones de aleatoriedad para cada escenario (rangos de datos, configuración del dispositivo/ambiente, tipos y pesos de errores, retardos) y decide cuándo la corrida termina. Implementado en <code>tests/test_general.sv</code> (aleatorio) y <code>tests/test_corner.sv</code> (dirigido, casos de esquina), ambos apoyados en <code>tests/base_test.sv</code> .
<code>generator</code>	Genera la secuencia de paquetes que conforman un escenario de prueba, repartiendo por turnos entre terminales para que varias envíen a la vez y el árbitro deba decidir. Clase <code>generator</code> (<code>tb/generator/generator.sv</code>).
<code>agent_drv</code> (padre) / <code>driver</code> (hijos)	El padre reparte cada paquete al hijo correspondiente según su campo <code>origen</code> y aplica el reset del DUT; cada hijo emula la FIFO de entrada de un terminal, levanta <code>pndng</code> y entrega el dato en <code>D_pop</code> , y marca <code>t_envio</code> cuando el DUT hace <code>pop</code> . Clases <code>agent_drv</code> y <code>driver</code> (<code>tb/driver/</code>).

Componente	Función
agent_mon (padre) / monitor (hijos)	Cada hijo vigila push/D_push de un terminal y arma un pkg_mon con lo observado; el padre reúne lo que ven todos los hijos hacia un solo mailbox de salida. Clases agent_mon y monitor (tb/monitor/).
scoreboard	Modelo de referencia: por cada paquete que el driver confirma que el DUT tomó, calcula quién(es) deberían recibirlo (unicast, broadcast o descartado) y guarda una ficha en espera. También concentra el reporte final y la escritura del CSV. Clase scoreboard (tb/scoreboard/scoreboard.sv).
checker (chequeador)	Por cada recepción observada por el monitor, pide al scoreboard que la empareje con una ficha en espera; si no hay ninguna, reporta una recepción inesperada. Lleva el conteo de errores del ambiente. Clase chequeador (tb/checker/checker.sv).
env	Instancia y conecta todos los bloques anteriores mediante los mailboxes correspondientes, y los arranca como procesos independientes. Clase env (tb/env/environment.sv).
bus_if	Interfaz SystemVerilog entre el ambiente y el DUT; sus señales tienen la misma forma que los puertos de bs_gnrtr_n_rbtr para poder conectarse directamente. (tb/interfaces/interfaz.sv).
tb_top	Módulo de más alto nivel: instancia el DUT, la interfaz y selecciona, mediante +prueba=general corner, cuál prueba correr. (tb/top/tb_top.sv).

3.2 Interfaz entre el ambiente y el DUT

```

1  'include "parametros.svh"
2
3
4  interface bus_if #(
5      parameter bits      = 'BITS,
6      parameter drvrs     = 'DRVRS,
7      parameter pkg_sz    = 'PKG_SZ
8  ) (
9      input bit clk
10 );
11
12 // las maneja el ambiente
13 logic          reset;
14 logic          pndng [bits-1:0][drvrs-1:0]; // hay dato en la fifo
15 logic [pkg_sz-1:0] D_pop [bits-1:0][drvrs-1:0]; // dato que va a sacar el DUT
16
17 // las maneja el DUT (wire porque vienen de puertos de salida)
18 wire          push [bits-1:0][drvrs-1:0]; // hay dato valido en D_push
19 wire          pop  [bits-1:0][drvrs-1:0]; // el DUT saco el dato
20 wire [pkg_sz-1:0] D_push [bits-1:0][drvrs-1:0]; // dato que llego
21
22 endinterface

```

Listing 1: Interfaz bus_if entre el ambiente y el DUT

4 Transacciones y estructuras de datos del ambiente

El ambiente define un paquete lógico de transacción por encima de las señales `D_push/D_pop` del RTL, con los siguientes campos:

Señal	Ancho / tipo	Descripción
<code>id_transaccion</code>	entero	Identificador único incremental de la transacción, usado para trazabilidad en el reporte CSV.
<code>d_origen</code>	8 bits	Dispositivo que origina el mensaje (ancho <code>ANCHO_DIR</code>).
<code>d_destino</code>	8 bits	Dispositivo destino, una dirección inexistente (para forzar mensajes con error) o el identificador de <i>broadcast</i> (<code>0xFF</code>). El ancho lo impone el DUT, que compara los 8 bits más altos del paquete contra el identificador de cada terminal.
<code>longitud</code>	16 32 64	Tamaño del paquete de datos, fijado en tiempo de compilación entre los tres valores soportados.
<code>payload</code>	longitud bits	Datos aleatorizados del paquete.
<code>tipo</code>	tipo_pckg	Enumerado que clasifica el envío: <code>envio_normal</code> , <code>envio_broadcast</code> , <code>envio_error</code> (dispositivo inexistente) o <code>envio_propio</code> . Gobierna la restricción que fija el destino (§6).
<code>t_envio</code>	tiempo (ns)	Marca de tiempo en que el driver inyecta el paquete al DUT.
<code>t_recibido</code>	tiempo (ns)	Marca de tiempo en que el monitor observa el paquete en el destino.

Estos campos se distribuyen en cuatro clases:

- **packet** – lo que genera el **generator** y consume el **driver**: incluye los campos aleatorizables y las restricciones (§6).
- **cfg** – la configuración que el **test** le envía al **generator** (cuántas transacciones por terminal, rangos de retardo, pesos de cada tipo de envío).
- **pckg_mon** – lo que el **monitor** observa en el lado de salida del DUT y le envía al **checker**.
- **trans_sb** – el registro que usa el **scoreboard** para unir lo que envió el driver con lo que confirmó el monitor, y que alimenta directamente el reporte CSV.

5 Ejecución concurrente del ambiente

Cada unidad del ambiente (**test**, **generator**, **agent/driver** por dispositivo, **agent/monitor** por dispositivo, **scoreboard** y **checker**) se ejecuta en un proceso independiente, sincronizados mediante eventos y mailboxes/colas. Esto se implementa en dos niveles:

1. **Nivel ambiente**: la clase `env` arranca los cinco bloques principales con `fork join_none`, de modo que corren de forma concurrente durante toda la simulación.
2. **Nivel por terminal**: tanto `agent_drv` como `agent_mon` instancian un hijo (`driver` / `monitor` respectivamente) por cada uno de los `drvrs` dispositivos, y cada hijo corre a su vez en su

propio proceso (`fork join_none` dentro de `driver::run()`, y un `fork` por hijo dentro de `agent_mon::run()`). Así, con `drvrs=4` hay simultáneamente 4 procesos de driver y 4 de monitor, además del generator, el scoreboard y el checker.

Mailbox	Conecta
<code>tst_gen_mbx (cfg_mbx)</code>	<code>test → generator</code>
<code>tst_agnt_mbx (escenario_mbx)</code>	<code>test → agente</code>
<code>gen_agnt_mbx (packet_mbx)</code>	<code>generator → agent_drv</code>
<code>buzon[i] (packet_mbx)</code>	<code>agent_drv (padre) → driver hijo i</code>
<code>drv_chkr_mbx (packet_mbx)</code>	<code>driver (todos los hijos) → scoreboard</code>
<code>buzon[i] (pckg_mon_mbx)</code>	<code>monitor hijo i → agent_mon (padre)</code>
<code>mon_chkr_mbx (pckg_mon_mbx)</code>	<code>agent_mon → checker</code>

Tabla 4: Mailboxes tipados (definidos en `tb/pkg/tb_pkg.sv`) que sincronizan los procesos del ambiente.

6 Generador y aleatorización

La aleatorización controlada es el núcleo del ambiente. Se implementa en dos niveles: la clase `cfg` aleatoriza *cuánto* tráfico genera cada terminal, y la clase `packet` aleatoriza *cada transacción individual*.

6.1 Número de transacciones por terminal

`cfg::trans_x_terminal[`DRVRS]` es un arreglo aleatorizado, uno por terminal, acotado por `min_trans/max_trans`:

```

1 rand int trans_x_terminal[`DRVRS];
2 constraint c_trans {
3   foreach (trans_x_terminal[i])
4     trans_x_terminal[i] inside {[min_trans:max_trans]};
5 }

```

De esta forma cada terminal envía una cantidad distinta de paquetes y el tráfico queda desbalanceado. Estos límites, junto con los rangos de retardo y los pesos de cada tipo de envío, se pueden ajustar sin recompilar mediante `plusargs` en `tests/test_general.sv` (`+min_trans`, `+max_trans`, `+min_delay`, `+max_delay`, `+p_nrml`, `+p_bdcst`, `+p_err`, `+p_prp`).

6.2 Largo del paquete (16, 32, 64)

El ancho del paquete (`pckg_sz`) es un parámetro estructural del hardware: no se aleatoriza en tiempo de corrida. El caso general se ejecuta en tres iteraciones de compilación independientes (configurando el DUT y el ambiente para 16, 32 y 64 bits respectivamente), resolviendo así el requerimiento de cobertura para los tres anchos soportados. `scripts/run.sh` escoge uno al azar (`shuf -n1 -e 16 32 64`) cuando no se indica explícitamente.

6.3 Tiempos de envío / retardo entre mensajes

Cada `packet` aleatoriza su propio retardo, en ciclos de reloj, antes de ser encolado en la FIFO emulada del driver:

```

1 rand int retardo; // ciclos antes de enviarlo
2 constraint c_retardo {retardo inside {[min_retardo:max_retardo]}};

```

6.4 Mensajes con error (destinos inexistentes)

El tipo de envío se aleatoriza con una distribución ponderada (`dist`), y el destino queda condicionado al tipo mediante restricciones dependientes:

```

1 constraint c_tipo {
2     tipo dist {envio_normal      := peso_normal,
3                envio_broadcast   := peso_broadcast,
4                envio_error       := peso_error,
5                envio_propio      := peso_propio};
6 }
7 constraint c_destino {
8     (tipo == envio_normal)      -> (destino < DRV_B && destino != origen);
9     (tipo == envio_broadcast)   -> (destino == 'BROADCAST');
10    (tipo == envio_error)        -> (destino >= DRV_B && destino != 'BROADCAST');
11    (tipo == envio_propio)       -> (destino == origen);
12 }

```

Los pesos por defecto (80 % normal, 10 % broadcast, 10 % error, 0 % propio) se configuran desde `cfg` y se pueden variar por `plusargs`.

6.5 Direcciones de destino y origen

El origen siempre corresponde a un dispositivo existente (`constraint c_origen {origen < DRV_B;}`), y en el `generator` se fija además al terminal que le corresponde el turno (`item.randomize() with {origen == i;}`), de modo que el reparto de tráfico entre terminales es controlado mientras que el resto de los campos (destino válido, tipo, retardo, payload) queda libre. El destino, por su parte, queda determinado por el tipo de envío como se mostró arriba: unicast válido distinto del origen, inexistente (`>= DRVRS`), o el propio origen.

6.6 Identificador de *broadcast*

Cuando `tipo == envio_broadcast`, el destino se fija en el valor reservado `'BROADCAST (8'hFF)`, que el DUT reconoce internamente comparando contra `{8{1'b1}}`. El scoreboard modela que un broadcast debe ser recibido por todos los terminales salvo el emisor (§10).

6.7 Generador: reparto por turnos

El `generator` no agota primero todas las transacciones de un terminal antes de pasar al siguiente: recorre los terminales por turnos, para que en la práctica varios terminales tengan tráfico pendiente al mismo tiempo y el árbitro del DUT deba resolver contención real.

code/tb/generator/generator.sv

```

1
2 class generator;
3
4     cfg_mbx      tst_gen_mbx; // config, viene del test
5     packet_mbx   gen_agnt_mbx; // los paquetes, van al agente
6
7     int generados = 0;
8     bit terminado = 0; // el test lo espera para saber cuándo parar

```

```

9
10 task run();
11     cfg      config_item;
12     packet  item;
13     int      faltan[];
14     int      quedan;
15
16     tst_gen_mbx.get(config_item);    // primero espera la config
17
18     faltan = new['DRVRS'];
19     foreach (faltan[i]) faltan[i] = config_item.trans_x_terminal[i];
20     quedan = config_item.total();
21
22     $display("[%0t] generator: va a generar %0d paquetes", $time, quedan);
23
24     while (quedan > 0) begin
25         foreach (faltan[i]) begin    // una vuelta por todas las terminales
26             if (faltan[i] > 0) begin // ¿a esta todavía le quedan?
27                 item = new();
28
29                 // los controles de la aleatorización salen de la cfg
30                 item.min_retardo    = config_item.min_delay;
31                 item.max_retardo    = config_item.max_delay;
32                 item.peso_normal    = config_item.peso_normal;
33                 item.peso_broadcast = config_item.peso_broadcast;
34                 item.peso_error     = config_item.peso_error;
35                 item.peso_propio    = config_item.peso_propio;
36
37                 // el origen lo fija el turno, el resto se aleatoriza
38                 if (!item.randomize() with {origen == i;}) begin
39                     $display("[%0t] generator: fallo randomize en el paquete %0d",
40                             $time, generados+1);
41                     $finish;
42                 end
43
44                 item.id = generados + 1;
45                 generados++;
46                 gen_agnt_mbx.put(item);
47
48                 faltan[i]--;
49                 quedan--;
50             end
51         end
52     end
53
54     terminado = 1;
55     $display("[%0t] generator: genero %0d paquetes", $time, generados);
56 endtask
57
58 endclass

```

7 Driver y Monitor

Driver siendo el que ejecuta un estímulo hacia el DUT y el monitor quien observa del DUT, cada uno con un padre que administra un hijo por terminal.

code/tb/driver/driver.sv

```

1
2 class driver;
3
4     virtual bus_if vif;
5

```

```

6  int id;                // terminal que emula
7
8  packet_mbx agnt_drv_mbx; // del padre
9  packet_mbx drv_chkr_mbx; // al checker
10
11 packet cola[$];        // la fifo emulada, es infinita: no tiene tope
12 int enviados          = 0;
13 int pops_vacios       = 0; // el DUT hizo pop sin que hubiera nada
14 int coincidencias     = 0; // entro un dato justo cuando salia otro
15
16 time t_ultimo_push = -1; // cuando se encolo el ultimo
17 time t_ultimo_pop  = -1; // cuando salio el ultimo
18
19 function new(int identificador = 0);
20     this.id = identificador;
21 endfunction
22
23 ...
24 begin
25     // esquina: pop con la cola vacía. No debería pasar nunca,
26     // porque en ese caso pndng esta en 0. Si pasa, es bug del DUT.
27     pops_vacios++;
28     $display("[%0t] driver[%0d]: ERROR pop con la cola vacía", $time, id);
29 end
30 else if (vif.pop[0][id] == 1'b1) begin // el DUT ya lo tomo
31     t_ultimo_pop = $time;
32     p = cola.pop_front();
33     p.t_envio = $time;
34     enviados++;
35     drv_chkr_mbx.put(p);
36 end
37 end
38 endtask
39
40 task run();
41     fork
42         recibir();
43         manejar();
44     join_none
45 endtask
46
47 endclass

```

code/tb/driver/agent_drv.sv

```

1
2 class agent_drv;
3
4     virtual bus_if vif;
5
6     packet_mbx gen_agnt_mbx; // del generator
7     packet_mbx drv_chkr_mbx; // al checker, compartido por los hijos
8
9     driver      hijos[];
10    packet_mbx buzon[]; // un mailbox por hijo
11
12    int repartidos = 0;
13    int descartados = 0;
14
15    function new(); // le pasa a cada hijo lo que necesita
16        hijos = new['DRVRS'];
17        buzon = new['DRVRS'];
18        foreach (hijos[i]) begin
19            buzon[i] = new();
20            hijos[i] = new(i);
21        end

```

```

22     endfunction
23
24     ...
25
26     // esquina: el DUT nunca debería hacer pop con la cola vacía
27     function int total_pops_vacios();
28         int t = 0;
29         foreach (hijos[i]) t += hijos[i].pops_vacios;
30         return t;
31     endfunction
32
33     // esquina: veces que entro un dato justo cuando salía otro
34     function int total_coincidencias();
35         int t = 0;
36         foreach (hijos[i]) t += hijos[i].coincidencias;
37         return t;
38     endfunction
39
40 endclass

```

code/tb/monitor/monitor.sv

```

1
2 class monitor;
3
4     virtual bus_if vif;
5
6     int id; // terminal que vigila
7
8     pkg_mon_mbx mon_agnt_mbx; // al padre
9
10    int recibidos = 0; // cuántos paquetes vio este hijo en total
11
12    function new(int identificador = 0);
13        this.id = identificador;
14    endfunction
15
16    task vigilar();
17        pkg_mon m;
18        forever begin
19            @(posedge vif.clk);
20            if (vif.push[0][id] == 1'b1) begin // ¿el DUT justo levantó push?
21                m = new();
22                m.terminal = id;
23                {m.destino, m.payload} = vif.D_push[0][id]; // parte el dato que llegó en dos: los
24                    bits altos van a destino, el resto a payload.
25                m.t_recibido = $time;
26
27                recibidos++;
28                mon_agnt_mbx.put(m);
29                m.print($sformatf("monitor[%0d]: recibido", id));
30            end
31        end
32    endtask
33
34    task run();
35        fork
36            vigilar();
37        join_none
38    endtask
39 endclass

```

8 Scoreboard y Checker

El **scoreboard** actúa como modelo de referencia: por cada paquete confirmado por el driver, calcula quién deberían recibirlo (según el campo **destino**, no según la etiqueta **tipo**, para no revisar el DUT contra la propia clasificación del ambiente) y mantiene una cola de fichas “esperando”. El **checker** recibe cada recepción observada por el monitor y le pide al scoreboard que la empareje contra esa cola, lo que nunca se empareja se reporta como perdido, y una recepción sin ficha que le calce se reporta como inesperada.

Como el origen no viaja por el bus (§2.3), ese emparejamiento se hace por contenido: terminal, destino y *payload*. Si dos terminales distintas enviaran la misma palabra al mismo destino y ambas estuvieran en vuelo a la vez, el scoreboard no podría distinguirlas y podría atribuir la recepción a la ficha equivocada. En lugar de modificar el formato del paquete para impedirlo, se optó por medirlo: el contador **ambiguos** se incrementa cada vez que más de una ficha calza con la misma recepción. En todas las corridas realizadas, incluidas las de más de 350 transacciones, ese contador dio 0.

code/tb/scoreboard/scoreboard.sv

```

1
2 class scoreboard;
3
4     packet_mbx drv_sb_mbx;    // del driver: lo que el DUT ya tomo
5
6     trans_sb esperando_q[$]; // todavía no llegan
7     trans_sb cerrados_q[$];  // entregados o descartados
8
9     int enviados      = 0;
10    int entregados    = 0;
11    int descartados    = 0;
12    int ambiguos      = 0;    // cuando dos registros calzan con la misma recepcion
13
14    string archivo = "reporte.csv";
15
16    task run(); // scoreboard lo recibe y anota
17        packet p;
18        forever begin
19            drv_sb_mbx.get(p);
20            anotar(p);
21        end
22    endtask
23
24    // un paquete solo llega si el destino existe y no es el mismo origen
25    function bit deberia_recibir(packet p, int i);
26        if (p.destino == 'BROADCAST) return (i != p.origen);
27        if (p.destino < 'DRVRS && p.destino != p.origen) return (i == p.destino);
28        return 0;
29    endfunction
30
31    function void anotar(packet p);
32        trans_sb r;
33        int cuantos = 0;
34
35        enviados++;
36
37        for (int i = 0; i < 'DRVRS; i++) begin // recorre los 4 terminales y por cada uno que "
38            debería recibirlo", crea una ficha nueva y la mete en esperando_q
39            if (deberia_recibir(p, i)) begin
40                r = new();
41                r.id      = p.id;
42                r.origen  = p.origen;
43                r.destino = p.destino;
44                r.terminal = i;

```

```

44     r.payload = p.payload;
45     r.t_envio = p.t_envio / 1ns;
46     r.estado = esperando;
47     esperando_q.push_back(r);
48     cuantos++;
49     end
50 end
51
52 if (cuantos == 0) begin // destino inexistente o a si mismo
53     r = new();
54     r.id = p.id;
55     r.origen = p.origen;
56     r.destino = p.destino;
57     r.terminal = -1;
58     r.payload = p.payload;
59     r.t_envio = p.t_envio / 1ns;
60     r.estado = descartado;
61     cerrados_q.push_back(r);
62     descartados++;
63 end
64 endfunction
65
66 ...
67
68 $display("---- scoreboard ----");
69 $display("paquetes enviados      : %0d", enviados);
70 $display("entregas esperadas      : %0d", entregados + perdidos());
71 $display("entregas confirmadas    : %0d", entregados);
72 $display("descartados a proposito : %0d", descartados);
73 $display("perdidos (no llegaron)   : %0d", perdidos());
74 $display("emparejamientos ambiguos : %0d", ambiguos);
75 if (entregados > 0)
76     $display("retraso min/prom/max      : %0d / %0d / %0d ns",
77             mn, suma/entregados, mx);
78
79 foreach (esperando_q[k])
80     esperando_q[k].print("scoreboard: ERROR nunca llego");
81 endfunction
82
83 endclass

```

code/tb/checker/checker.sv

```

1
2 class chequeador;
3
4     pkg_mon_mbx mon_chkr_mbx; // del monitor
5     scoreboard sb;
6
7     int vistos = 0;
8     int sin_esperar = 0;
9
10    task run();
11        pkg_mon m;
12        forever begin
13            mon_chkr_mbx.get(m); // extraemos dato del monitor
14            vistos++;
15            if (!sb.emparejar(m)) begin // comparamos
16                sin_esperar++;
17                m.print("checker: ERROR nadie esperaba esta recepcion");
18            end
19        end
20    endtask
21
22    function void reporte();
23        $display("---- checker ----");

```

```

24     $display("recepciones vistas      : %0d", vistos);
25     $display("recepciones inesperadas : %0d", sin_esperar);
26     endfunction
27
28     function int errores();
29         return sin_esperar + sb.perdidos();
30     endfunction
31
32 endclass

```

9 Casos de esquina

Además del caso general (tráfico aleatorio concurrente en los M dispositivos), se identificaron e implementaron los siguientes casos de esquina como pruebas dirigidas en `tests/test_corner.sv`. Cada caso inyecta paquetes armados a mano directamente al agente (sin pasar por el generador), y se verifica con el mismo scoreboard/checker del ambiente.

Caso	Descripción y motivación
1. Todos los dispositivos solicitando el bus en el mismo ciclo	Se varía el número de solicitantes simultáneos (de 2 hasta M); expone bugs de prioridad fija o <i>starvation</i> en el árbitro que con tráfico normal podrían no manifestarse nunca.
2. Mensajes consecutivos con error (destino inexistente) desde el mismo terminal	Se varía la longitud de la ráfaga de errores seguidos (3, 5, 10); confirma que el manejo de errores no acumula estado ni degrada el desempeño del bus con el tiempo.
3. Transacción con el mínimo y el máximo retardo permitido entre mensajes	Se fija el retardo exactamente en los límites del rango (no aleatorio dentro de él); los bugs de temporización casi siempre aparecen en los bordes, no en el promedio, y el random puro rara vez los toca.
4. FIFO de un dispositivo vacía y se solicita <code>pop</code> a pesar de ello	Propiedad vigilada dentro del propio <code>driver</code> (no se inyecta): si no se maneja el <i>underflow</i> , el diseño podría entregar datos basura como si fueran válidos, generando falsos positivos silenciosos.
5. Patrón de datos alternante (<i>checkerboard</i>) en el payload	Se varía el ancho del patrón según la longitud del paquete (16/32/64) y cuántas transacciones consecutivas lo mantienen; detecta acoplamiento entre bits adyacentes que pasa desapercibido con datos puramente aleatorios, sobre todo al pasar por los serializadores.
6. Direcciones justo en los bordes del rango válido	Se prueban destino 0, <code>drvrs-1</code> , <code>drvrs</code> (primer inexistente), <code>0xFE</code> y <code>0xFF</code> (broadcast); el DUT decide con una comparación de 8 bits, y los bordes son donde se ven los errores de $<$ contra $\leq$.
7. Todos los terminales mandando al mismo destino	El caso contrario al broadcast (en vez de uno a todos, todos a uno); ejercita la cola de recepción de un solo terminal bajo presión sostenida desde múltiples emisores.
8. Actividad alternante entre pares de terminales	Se varía el período de alternancia y qué par de dispositivos se alterna; es un patrón dirigido que el random puro difícilmente genera y expone problemas de sincronización que solo aparecen con cambios de actividad repetitivos y predecibles.

Caso	Descripción y motivación
9. Múltiples dispositivos operando simultáneamente, mezclando tipos de envío	Se varía cuántos dispositivos actúan a la vez y si sus operaciones son del mismo tipo o mixtas (normal / broadcast / error); prueba que el árbitro y las FIFOs manejan la concurrencia real del sistema, no solo actividad de a un dispositivo por vez.
10. Datos entrando a una FIFO mientras el DUT saca de ella (<i>push</i> y <i>pop</i> concurrentes)	Se llena primero un poco la cola y luego se alimenta “a goteo” mientras el DUT sigue sacando; expone condiciones de carrera clásicas en FIFOs que pueden perder o duplicar datos sin que sea evidente en pruebas simples.

Tabla 5: Casos de esquina identificados e implementados en `test_corner.sv` (los casos 1–3 y 5–10 se inyectan como estímulo; el caso 4 se vigila como propiedad).

Al final de la corrida de esquina se reportan explícitamente dos métricas adicionales que resumen el cumplimiento de las propiedades vigiladas: la cantidad de veces que el DUT hizo `pop` con la cola vacía (debe ser 0) y la cantidad de coincidencias entre una entrada y una salida en el mismo ciclo (caso 10).

code/tests/test_corner.sv

```

1 class test_corner extends base_test;
2
3 localparam int PAYLOAD = 'PCKG_SZ - 'ANCHO_DIR;
4
5
6 int max_delay = 10; // se usa en el caso 3
7
8 //////////////////////////////////////////////////
9 // 1. varios terminales piden el bus a la vez
10 // se repite con 2, 3 ... hasta DRVRS solicitantes
11 //////////////////////////////////////////////////
12 task caso_simultaneos();
13     packet p;
14     for (int n = 2; n <= 'DRVRS; n++) begin
15         $display("\n[%0t] === esquina 1: %0d terminales piden el bus a la vez ===",
16             $time, n);
17         for (int d = 0; d < n; d++) begin
18             // retardo 0 para que entren todos en el mismo ciclo
19             p = armar(d, (d+1) % 'DRVRS, 'hAO + d, 0, envio_normal);
20             inyectar(p);
21         end
22         esperar_fin();
23     end
24 endtask
25
26 //////////////////////////////////////////////////
27 // 2. mensajes con error seguidos desde el mismo terminal
28 // se varia el largo de la rafaga: 3, 5 y 10
29 //////////////////////////////////////////////////
30 task caso_rafaga_errores();
31     packet p;
32     int largos[3] = '{3, 5, 10};
33
34     foreach (largos[k]) begin
35         $display("\n[%0t] === esquina 2: rafaga de %0d errores seguidos ===",
36             $time, largos[k]);
37         for (int i = 0; i < largos[k]; i++) begin

```

```

38      // destinos que no existen, pero distintos del broadcast
39      p = armar(0, 'DRVRS + i, 'h5A, 0, envio_error);
40      injectar(p);
41      end
42      // uno bueno al final, para ver que el bus sigue sirviendo
43      p = armar(0, 1, 'h77, 0, envio_normal);
44      injectar(p);
45      esperar_fin();
46      end
47      endtask
48
49      ...
50
51      task run();
52      void'($value$plusargs("max_delay=%d", max_delay));
53
54      arrancar();
55      $display("[%0t] test_corner: PCKG_SZ=%0d DRVRS=%0d", $time, 'PCKG_SZ, 'DRVRS);
56
57      caso_simultaneos();
58      caso_rafagaErrores();
59      caso_retardos_borde();
60      caso_checkerboard();
61      caso_bordes_direccion();
62      caso_todos_a_uno();
63      caso_alternante();
64      caso_mixto();
65      caso_push_con_pop();
66
67      $display("\n---- resumen de los casos de esquina ----");
68      $display("paquetes inyectados      : %0d", sig_id);
69      $display("pop con la cola vacía      : %0d (debe ser 0)",
70              e0.agnt_drv0.total_pops_vacios());
71      $display("push y pop en el mismo ciclo : %0d",
72              e0.agnt_drv0.total_coincidencias());
73      reporte_final();
74      endtask
75
76      endclass

```

10 Generación de resultados: reporte CSV e histograma

Al final de cada corrida, el `scoreboard` escribe un archivo CSV con una línea por cada ficha de transacción cerrada (entregada, descartada o perdida):

```

1 static function string encabezado_csv();
2     return "id_transaccion,t_envio,d_origen,d_destino,terminal,t_recibido,retraso";
3 endfunction

```

id_transaccion	t_envio	d_origen	d_destino	terminal	t_recibido	retraso
2	145	1	3	3	335	190
3	165	2	1	1	535	370
4	245	3	0	0	735	490
8	765	3	225	-1		
1	165	0	1	1	935	770
10	1165	1	255	0	1935	770
10	1165	1	255	2	1935	770

Tabla 6: Extracto de `docs/reporte_ejemplo.csv`. La fila con `terminal=-1` y sin `t_recibido/retraso` corresponde a un mensaje descartado (destino 225, inexistente para `drvrs=4`); las tres filas con `id_transaccion=10` y mismo `t_recibido` corresponden a un mismo *broadcast* recibido por tres terminales distintas.

A partir de ese CSV, `scripts/plot_histograma.gnu` genera con GNUplot un histograma del retraso de los paquetes recibidos (columna 7), agrupando en barras de 50 ns por defecto:

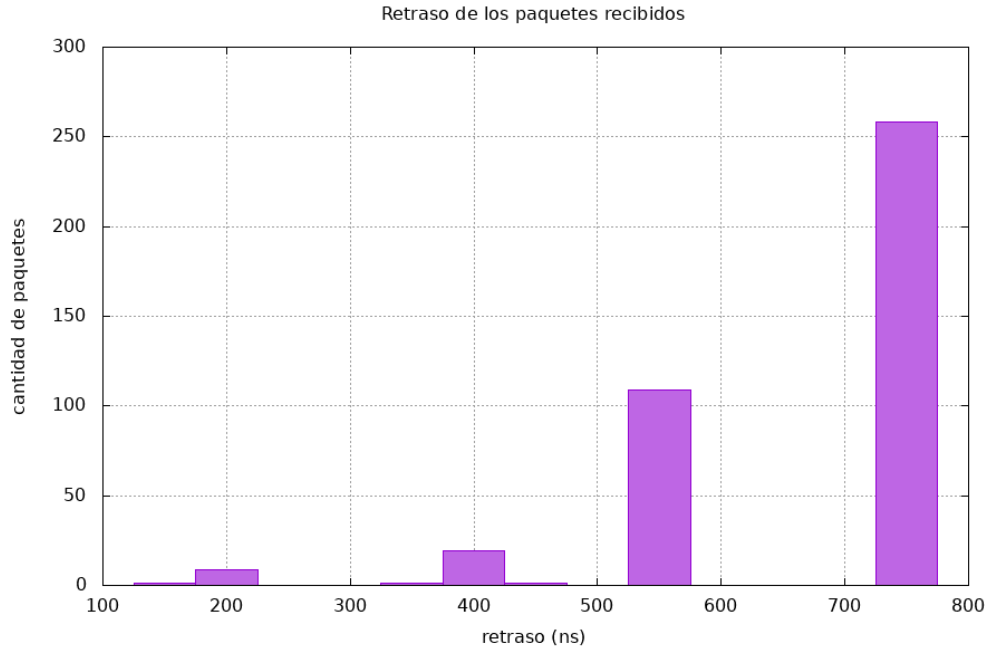


Figura 2: Histograma de ejemplo generado con `plot_histograma.gnu` sobre `reporte_ejemplo.csv`: distribución del retraso (ns) de los paquetes recibidos.

10.1 Resultados de una corrida real y verificación

Como evidencia de que el ambiente funciona sobre el DUT real (no solo sobre el ejemplo anterior), se ejecutó `scripts/run.sh` con `drvrs=8`, lo que deja en `sim/` los tres reportes de la tabla 7: la corrida general, la de casos de esquina, y una corrida larga (`+min_trans=40 +max_trans=110 +max_delay=15`) usada para alimentar el histograma.

- que todo *broadcast* llegue a exactamente `drvrs-1 = 7` terminales (ni de más ni de menos);
- que toda transacción *unicast* sin entrega confirmada corresponda a un `envio_error` (destino inexistente), que por diseño nunca debería entregarse, y no a una pérdida sin explicación;

- que no existan entregas duplicadas (mismo `id_transaccion` y `terminal` repetido) ni entregas a un terminal distinto del `d_destino` declarado;
- que $\text{retraso} = t_{\text{recibido}} - t_{\text{envio}}$ se cumpla en el 100 % de las filas con entrega confirmada.

Reporte	Trans.	Broadcast	Unicast s/entrega	¿Todas envío_error?
reporte_general.csv	58	12/12	4	Sí
reporte_corner.csv	340	4/4	22	Sí
reporte.csv (largo)	701	73/73	60	Sí

Tabla 7: Resumen de las tres fases de una corrida real (`drvrs=8`). “Broadcast” es la fracción de transacciones *broadcast* que llegaron a los `drvrs-1 = 7` terminales esperados. En las tres fases, retraso, duplicados y destino de entrega se verificaron sin ninguna inconsistencia.

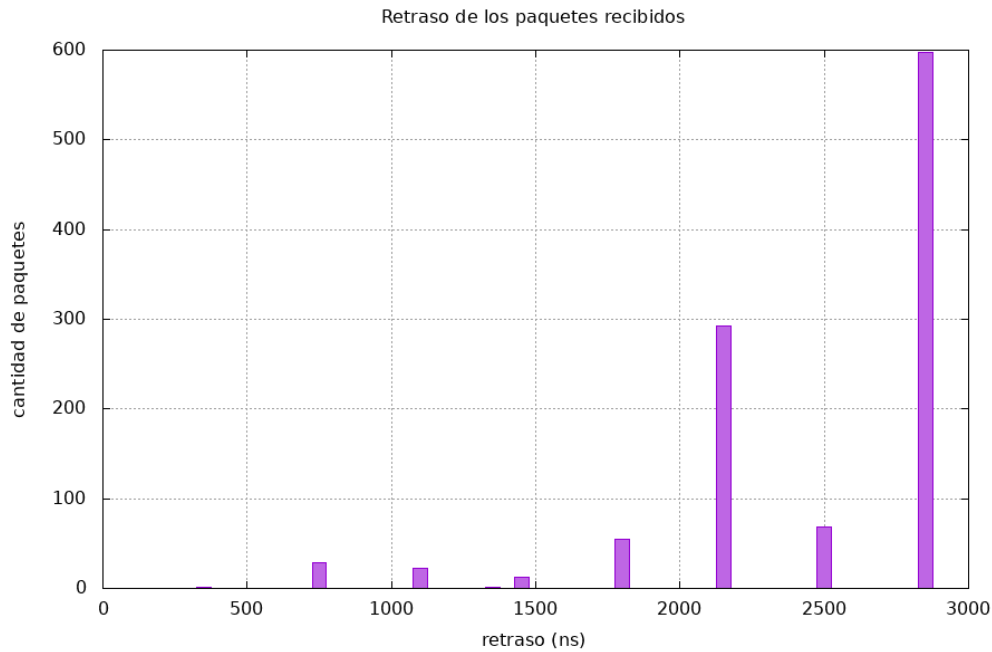


Figura 3: Histograma real (`sim/histograma.png`) de la corrida larga con `drvrs=8`. La acumulación grande cerca de 2850 ns corresponde al retraso de estado estacionario al que convergen los paquetes cuando el bus se satura por el `+max_delay=15` (envíos muy seguidos, poco tiempo de espera entre ellos).

Ese valor de saturación no es arbitrario: corresponde a una vuelta completa del árbitro. Como el DUT hace `pop` apenas ve `pndng` y transmite un paquete a la vez, un terminal nunca espera más de lo que tardan los otros `drvrs-1` en usar el bus. Por eso el retraso máximo queda acotado por la rotación y no crece con la carga: al aumentar las transacciones por terminal el promedio sube, pero el máximo se mantiene en el mismo valor.

10.2 Cobertura de los tres anchos de paquete (16, 32 y 64 bits)

Siguiendo §6 (`pckg_sz` se resuelve por compilación independiente, no en tiempo de corrida), se repitió `scripts/run.sh` una vez por cada ancho (`pckg_sz` = 16, 32, 64), siempre con `drvrs` = 8. Las tres compilaciones pasan exactamente los mismos chequeos de la tabla 7:

<code>pckg_sz</code>	Reporte	Trans.	Broadcast	Unicast s/entrega	¿Todas envio_error?
16	general	58	12/12	4	Sí
	corner	340	4/4	22	Sí
	largo	701	73/73	60	Sí
32	general	58	12/12	4	Sí
	corner	340	4/4	22	Sí
	largo	701	73/73	60	Sí
64	general	58	12/12	4	Sí
	corner	340	4/4	22	Sí
	largo	701	73/73	60	Sí

Tabla 8: Mismos chequeos de la tabla 7 aplicados a las tres compilaciones independientes (`drvrs`=8). Las tres pasan sin inconsistencias.

Las tres compilaciones producen exactamente el mismo patrón de tráfico porque la semilla por defecto de VCS no depende de los `+define` de compilación. Lo único que cambia entre ellas es el ancho del paquete en las señales `D_push/D_pop`, que no queda registrado en este CSV. No invalida la cobertura estructural de los tres anchos, pero significa que las tres corridas no son muestras aleatorias independientes entre sí (§10.3).

10.3 Semilla de aleatorización: fija vs. variable

Para confirmar en la práctica que la aleatorización entre corridas es real (y no solo que el código tiene `rand`), se corrió dos veces el mismo binario ya compilado (`./salida`, sin recompilar) bajo dos condiciones:

- **Sin especificar semilla:** dos corridas seguidas (`reporte_seed_fija_{a,b}.csv`) dieron un archivo **idéntico byte a byte** (`diff` sin salida) lo que confirma que VCS usa su semilla fija por defecto cuando no se pasa `+ntb_random_seed`.
- **Con `+ntb_random_seed=$RANDOM`:** dos corridas (`reporte_seed_variable_{a,b}.csv`) dieron **148 líneas distintas** sobre un total de unas 75 cada una, es decir, casi todo el tráfico generado (quién le manda a quién, cuándo, con qué retraso) cambió entre una corrida y la otra.

Esto confirma lo que ya se sabía por el código: sin `+ntb_random_seed` explícito, corridas repetidas con los mismos parámetros producen exactamente los mismos datos “aleatorios”. Como extensión natural, el mismo barrido de `pckg_sz` de §10.2 podría repetirse pasando una semilla distinta en cada compilación, para sumar a la cobertura estructural (los tres anchos) una muestra de tráfico realmente independiente por ancho.

10.4 Matriz completa: los tres anchos cruzados con tres tamaños de bus

La cobertura de ancho de paquete de la §10.2 se hizo con `drvrs`=8 fijo. Para confirmar que el ambiente también escala correctamente en la otra dimensión estructural, la cantidad de dispositivos

en el bus, fijada por `+define+DRVRS` al compilar – se repitió `scripts/run.sh` para las nueve combinaciones de `pckg_sz` $\in \{16, 32, 64\}$ y `drvrs` $\in \{4, 8, 16\}$, sin modificar ningún otro parámetro. A diferencia de la corrida de la §10.1, esta vez sí se conservó el `.log` completo de cada compilación (§??).

<code>pckg_sz</code>	<code>drvrs</code>	Trans. general	Trans. esquina	Trans. largo	PASS
16	4	30	170	353	Sí
16	8	58	340	701	Sí
16	16	109	1016	1351	Sí
32	4	30	170	353	Sí
32	8	58	340	701	Sí
32	16	109	1016	1351	Sí
64	4	30	170	353	Sí
64	8	58	340	701	Sí
64	16	109	1016	1351	Sí

Tabla 9: Nueve compilaciones independientes (una por combinación de `pckg_sz` y `drvrs`), cada una ejecutando las tres fases de `scripts/run.sh`. “Trans. esquina” es el conteo de paquetes inyectados por `test_corner.sv`, que depende solo de `drvrs` por ser un estímulo dirigido y no aleatorio.

Como era de esperar, el conteo de transacciones de las fases general y de esquina depende únicamente de `drvrs` (más dispositivos, más turnos de reparto) y no de `pckg_sz`: cada ancho produce exactamente el mismo número de paquetes, consistente con el hallazgo de la §10.2 de que la semilla por defecto de VCS no depende de los `+define`. Donde sí cambia el comportamiento con la cantidad de terminales es en el tiempo que tarda el bus en entregar cada paquete: las figuras 4 y 5 comparan el histograma de retraso para `drvrs=4` contra `drvrs=16` (ambos con `pckg_sz=16`). El pico de estado estacionario se desplaza de unos 750 ns con 4 terminales a más de 3000 ns con 16, porque hay más dispositivos compitiendo por el mismo bus compartido.

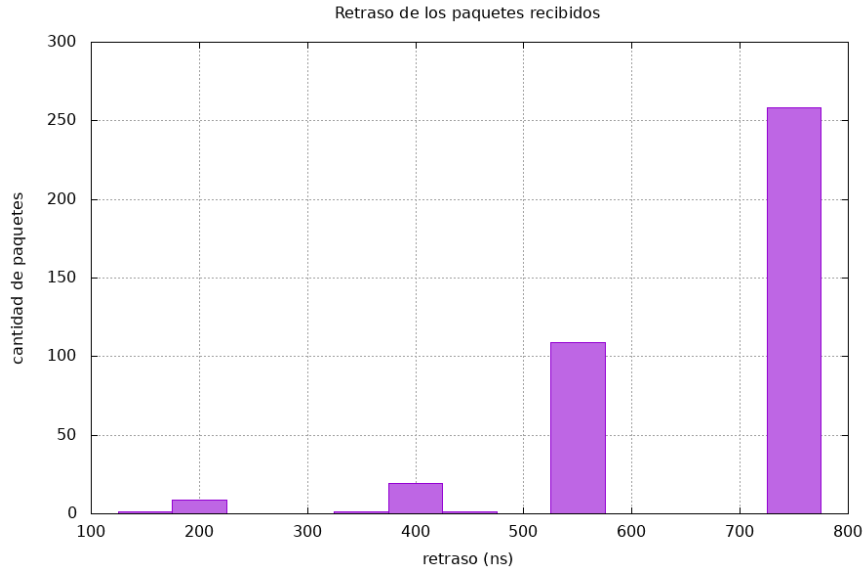


Figura 4: Retraso de paquetes con `pckg_sz=16`, `drvrs=4` (corrida larga de la matriz, §10.4). Pico de estado estacionario cerca de 750 ns.

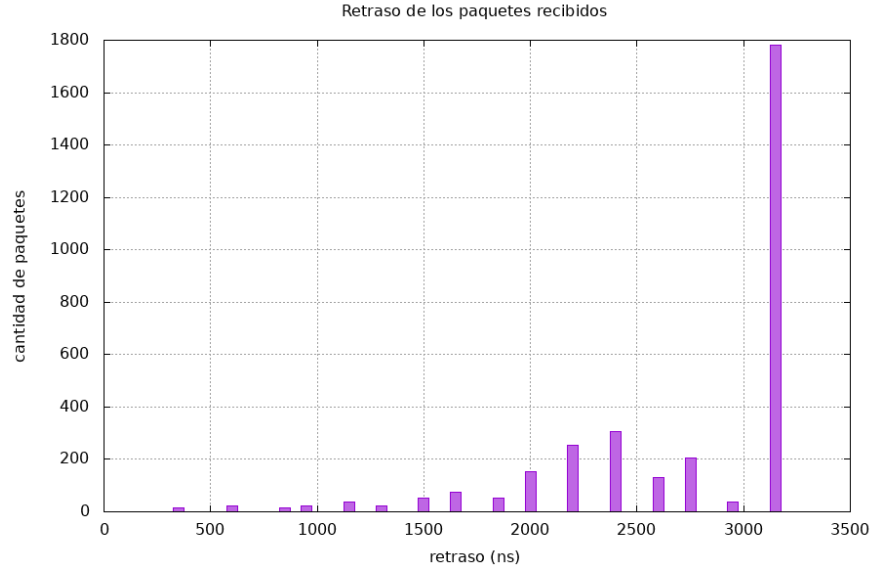


Figura 5: Retraso de paquetes con `pckg_sz=16`, `drvrs=16` (misma corrida larga, §10.4). Con el cuádruple de dispositivos conteniendo por el mismo bus, el pico de estado estacionario se desplaza a más de 3000 ns.

10.5 Barrido dirigido de distribución y temporización de tráfico

Además de la corrida general con los pesos por defecto (§6), se corrieron seis variantes dirigidas sobre las nueve combinaciones estructurales, cambiando únicamente los `plusargs` de pesos y retardo (sin recompilar). El objetivo es ejercitar configuraciones extremas que el caso general, por construcción, visita con poca frecuencia.

Escenario	N/B/E/P (%)	Retardo (ns)	Gen.	Vistas	Perdidos	PASS
<code>con_propio</code>	70/10/10/10	0–8	58	84	0	Sí
<code>solo_bdcst</code>	0/100/0/0	0–8	58	406	0	Sí
<code>solo_error</code>	0/0/100/0	0–8	58	0	0	Sí
<code>delay_alto</code>	80/10/10/0	10–30	58	126	0	Sí
<code>delay_cero</code>	80/10/10/0	0–0	58	126	0	Sí
<code>muchas</code>	80/10/10/0	0–8	1343	2056	0	Sí

Tabla 10: Seis variantes dirigidas, mostrando la fila `pckg_sz=16`, `drvrs=8` de cada una (las otras ocho combinaciones estructurales dan los mismos conteos, salvo por la observación de la §10.6). N/B/E/P = normal/*broadcast*/error/*propio*.

En las 54 corridas de estas seis variantes, el checker reporta siempre 0 recepciones inesperadas y el scoreboard siempre 0 perdidos, incluso en los extremos de `solo_bdcst` y `solo_error` donde la distribución por defecto casi nunca llega. Adicionalmente, por cada una de las nueve combinaciones se corrió una séptima variante (*semilla*) pasando `+ntb_random_seed=$RANDOM`; los totales generados difieren entre combinaciones (de 19 a 100 transacciones) confirmando, igual que en la §10.3, que cada una recibe una semilla distinta.

10.6 Hallazgo: la coincidencia *push/pop* depende del ancho del paquete

El contador de coincidencias entre una entrada y una salida en el mismo ciclo (`driver::coincidencias`, caso de esquina 10 en la §9 cuando se fuerza explícitamente) también aparece de forma *natural* en tráfico aleatorio, y resultó depender del ancho del paquete de una forma que no era evidente antes de correr la matriz completa:

- En las fases **general** y **largo** (tráfico aleatorio, misma semilla en las tres compilaciones por §10.2), las coincidencias **solo aparecen con `pckg_sz=16`**: con `drvrs=16` la fase **largo** registra 30 coincidencias con `pckg_sz=16` y 0 con `pckg_sz=32` o `64`; la variante **muchas** (§10.5) llega a 49 coincidencias sobre 2615 paquetes con `pckg_sz=16`, `drvrs=16`, y 0 con los otros dos anchos.
- En la fase **esquina** (estímulo dirigido idéntico en los tres anchos, que fuerza *push* y *pop* a propósito), el conteo de “push y pop en el mismo ciclo” en cambio **crece con el ancho**: con `drvrs=4` pasa de 247 (`pckg_sz=16`) a 358 (32) y 582 (64).

Ambas observaciones son consistentes con que el **serializer** del DUT envía cada palabra bit a bit (§2): un paquete más ancho ocupa más ciclos de reloj en transferirse, lo que cambia la alineación exacta entre cuándo un terminal libera el bus y cuándo otro puede tomarlo, aun con la misma secuencia lógica de eventos. En ningún caso esto afecta el resultado: el scoreboard cuenta la coincidencia aparte (no la confunde con una pérdida ni con una entrega duplicada) y las 90 corridas de la matriz pasan igual. Se deja como observación porque ilustra que, aunque las tres compilaciones comparten el mismo flujo de datos de alto nivel (§10.2), no son idénticas a nivel de temporización de señal.

11 Cómo compilar y correr el ambiente

Los parámetros estructurales (`PCKG_SZ`, `DRVRS`) se fijan al compilar con VCS mediante `+define`. El script `scripts/correr.sh` centraliza esa compilación y corrida. El código completo, los resultados y la documentación se encuentran en el repositorio del proyecto: <https://github.com/GeraldBS/Proyecto-1-Verificaci-n-funcional>.

El script `scripts/run.sh` orquesta una corrida completa: compila y corre la prueba general, luego la de casos de esquina, y finalmente una corrida larga (más transacciones por terminal) para alimentar el histograma.

11.1 Pruebas unitarias dirigidas previas al ambiente completo

Antes de integrar el ambiente completo, se validaron por separado sus piezas principales con pruebas dirigidas sencillas, cada una ejecutable de forma independiente con `scripts/correr.sh <prueba>`:

- **prueba_humo**: envía un paquete de cada tipo (unicast, broadcast, error, auto-envío) directamente contra el DUT, sin el ambiente completo, para confirmar que la interfaz conecta correctamente antes de construir el resto.
- **prueba_driver**: usa el `agent_drv` real para verificar que reparte, encola y entrega los paquetes en el orden correcto, y que marca `t_envio`.
- **prueba_monitor**: agrega el `agent_mon` real sobre la anterior, y compara contra los mismos paquetes de `prueba_driver` para confirmar que el monitor ve exactamente lo que el DUT entregó.

Este enfoque incremental (interfaz → driver → monitor → ambiente completo) redujo el riesgo de depurar simultáneamente el DUT, el driver y el monitor durante la integración final.

12 Conclusión

Sobre una corrida real (`drvrs=8`) se verificó que las tres fases (general, esquina y la corrida larga para el histograma) son consistentes: todo *broadcast* llegó a los `drvrs-1` terminales esperados, ninguna transacción *unicast* válida quedó sin entregar, y no se encontraron entregas duplicadas ni mal dirigidas (§10.1).

Se completó además la cobertura de los tres anchos de paquete exigidos por el caso general (16, 32 y 64 bits, §10.2), cruzada con las tres cantidades de terminales soportadas (`drvrs = 4, 8, 16`, §10.4): nueve combinaciones estructurales en total, cada una verificada con el texto literal PASS del checker preservado en su `.log`. Sobre esa misma matriz se corrieron seis variantes dirigidas de distribución y temporización de tráfico, incluyendo los extremos de 100 % *broadcast*, 100 % error, retardo nulo y una prueba de volumen de más de 2600 transacciones, más una séptima variante con semilla explícita distinta por combinación, sumando 90 corridas sin una sola pérdida ni recepción inesperada (§10.5). También se confirmó de forma empírica que la aleatorización entre corridas es real: dos corridas sin semilla explícita dan resultados idénticos, y dos con `+ntb_random_seed=$RANDOM` dan tráfico genuinamente distinto (§10.3). La propia matriz expuso además una sutileza no anticipada: la temporización exacta de la coincidencia *push/pop* sí depende del ancho de paquete, aun cuando el flujo lógico de datos es idéntico entre anchos (§10.6).

A Anexo: código fuente completo del ambiente

Esta sección reúne, para referencia completa, el código fuente restante del ambiente que no se incluyó en el cuerpo del documento.

code/tb/parametros.svh

```

1 ////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
2 // Parametros que cambian el hardware, se fijan al compilar.                //
3 // ej: vcs ... +define+PKG_SZ=32 +define+DRVRS=8                            //
4 // Si no se pasan se usan los valores de aqui.                             //
5 ////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
6 #ifndef PARAMETROS_SVH
7 #define PARAMETROS_SVH
8
9 // tamaño del paquete (16, 32 o 64)
10 #ifndef PKG_SZ
11 #define PKG_SZ 16
12 #endif
13
14 // cantidad de dispositivos en el bus
15 // máximo 254 para poder mandar a destinos que no existen
16 // (las direcciones son de 8 bits y 0xFF es broadcast)
17 #ifndef DRVRS
18 #define DRVRS 4
19 #endif
20
21 // número de buses, el enunciado lo deja en 1
22 #ifndef BITS
23 #define BITS 1
24 #endif
25
26 // dirección de broadcast
27 // no cambiar: el DUT compara internamente contra {8{1'b1}}
28 #ifndef BROADCAST
29 #define BROADCAST 8'hFF
30 #endif
31

```

```

32 // bits de direccion dentro del paquete (el DUT usa los 8 mas altos)
33 `ifndef ANCHO_DIR
34     `define ANCHO_DIR 8
35 `endif
36
37 // periodo del reloj en ns
38 `ifndef PERIODO_CLK
39     `define PERIODO_CLK 10
40 `endif
41
42 `endif

```

Listing 2: Parámetros de compilación del ambiente (code/tb/parametros.svh)

B Anexo: pruebas unitarias dirigidas

code/tests/prueba_humo.sv

```

1  `timescale 1ns/1ps
2
3  // se define FIFOS para que Library.sv no busque ../FIFO_Latches/fifo.sv
4  `include "fifo.sv"
5  `define FIFOS
6  `include "Library.sv"
7
8  `include "parametros.svh"
9  `include "interfaces/interfaz.sv"
10 `include "pkg/tb_pkg.sv"
11
12 module prueba_humo;
13
14     parameter bits      = `BITS;
15     parameter drvrs     = `DRVRS;
16     parameter pkg_sz    = `PKG_SZ;
17
18     // tiempo de espera: serializar + una vuelta del arbitro + margen
19     localparam int CICLOS_ESPERA = pkg_sz + 2*drvrs + 20;
20
21     bit clk = 0;
22     always #(`PERIODO_CLK/2.0) clk = ~clk;
23
24     bus_if #(.bits(bits), .drvrs(drvrs), .pkg_sz(pkg_sz)) _if (.clk(clk));
25
26     bs_gnrtr_n_rbtr #(.bits(bits), .drvrs(drvrs), .pkg_sz(pkg_sz),
27         .broadcast(`BROADCAST)) dut (
28         .clk      (_if.clk),
29         .reset    (_if.reset),
30         .pndng    (_if.pndng),
31         .push     (_if.push),
32         .pop      (_if.pop),
33         .D_pop    (_if.D_pop),
34         .D_push   (_if.D_push)
35     );
36
37     ...
38
39     // destino que no existe
40     p = new();
41     p.tipo = envio_error;      p.origen = 0; p.destino = drvrs;
42     p.payload = 'h5A;         p.id = 3;
43     enviar_y_revisar(p);
44
45     // destino igual al origen
46     p = new();

```

```

47     p.tipo = envio_propio;    p.origen = 1; p.destino = 1;
48     p.payload = 'hC3;        p.id = 4;
49     enviar_y_revisar(p);
50
51     if (errores == 0) $display("==== PRUEBA DE HUMO: PASS ====");
52     else              $display("==== PRUEBA DE HUMO: FAIL (%0d errores) ====", errores);
53     $finish;
54 end
55
56 // por si el DUT nunca hace pop
57 initial begin
58     #('PERIODO_CLK * (4 * CICLOS_ESPERA + 200));
59     $display("==== PRUEBA DE HUMO: TIMEOUT ====");
60     $finish;
61 end
62
63 endmodule

```

code/tests/prueba_driver.sv

```

1  `timescale 1ns/1ps
2
3  `include "fifo.sv"
4  `define FIFOS
5  `include "Library.sv"
6
7  `include "parametros.svh"
8  `include "interfaces/interfaz.sv"
9  `include "pkg/tb_pkg.sv"
10
11 module prueba_driver;
12
13     parameter bits      = 'BITS;
14     parameter drvrs     = 'DRVRS;
15     parameter pckg_sz   = 'PCKG_SZ;
16
17     bit clk = 0;
18     always #('PERIODO_CLK/2.0) clk = ~clk;
19
20     bus_if #(.bits(bits), .drvrs(drvrs), .pckg_sz(pckg_sz)) _if (.clk(clk));
21
22     bs_gnrtr_n_rbtr #(.bits(bits), .drvrs(drvrs), .pckg_sz(pckg_sz),
23         .broadcast('BROADCAST)) dut (
24         .clk (_if.clk), .reset(_if.reset), .pndng(_if.pndng),
25         .push(_if.push), .pop (_if.pop),    .D_pop(_if.D_pop), .D_push(_if.D_push)
26     );
27
28     ...
29
30     if (padre.repartidos != esperados) begin
31         errores++;
32         $display("ERROR: el padre repartio %0d y se esperaban %0d", padre.repartidos,
33             esperados);
34     end
35     if (padre.total_enviados() != esperados) begin
36         errores++;
37         $display("ERROR: el DUT tomo %0d paquetes y se esperaban %0d", padre.total_enviados(),
38             esperados);
39     end
40
41     // revisa que llegaran al checker con t_envio
42     while (drv_chkr_mbx.num() > 0) begin
43         drv_chkr_mbx.get(p);
44         if (p.t_envio == 0) begin
45             errores++;
46             $display("ERROR: paquete id=%0d sin t_envio", p.id);

```

```

45     end
46 end
47
48 if (errores == 0) $display("==== PRUEBA DRIVER: PASS ====");
49 else             $display("==== PRUEBA DRIVER: FAIL (%0d errores) ====", errores);
50 $finish;
51 end
52
53 initial begin
54     #('PERIODO_CLK * 20000);
55     $display("==== PRUEBA DRIVER: TIMEOUT ====");
56     $finish;
57 end
58
59 endmodule

```

code/tests/prueba_monitor.sv

```

1  `timescale 1ns/1ps
2
3  `include "fifo.sv"
4  `define FIFOS
5  `include "Library.sv"
6
7  `include "parametros.svh"
8  `include "interfaces/interfaz.sv"
9  `include "pkg/tb_pkg.sv"
10
11 module prueba_monitor;
12
13     parameter bits      = 'BITS;
14     parameter drvrs     = 'DRVRS;
15     parameter pkg_sz    = 'PKG_SZ;
16
17     localparam int ANCHO_PAYLOAD = pkg_sz - 'ANCHO_DIR;
18
19     bit clk = 0;
20     always #('PERIODO_CLK/2.0) clk = ~clk;
21
22     bus_if #(.bits(bits), .drvrs(drvrs), .pkg_sz(pkg_sz)) _if (.clk(clk));
23
24     bs_gnrtr_n_rbtr #(.bits(bits), .drvrs(drvrs), .pkg_sz(pkg_sz),
25         .broadcast('BROADCAST)) dut (
26         .clk (_if.clk), .reset(_if.reset), .pndng(_if.pndng),
27         .push(_if.push), .pop (_if.pop),    .D_pop(_if.D_pop), .D_push(_if.D_push)
28     );
29
30     agent_drv     padre_drv;
31     agent_mon     padre_mon;
32     packet_mbx    gen_agnt_mbx;
33     packet_mbx    drv_chkr_mbx;
34     pkg_mon_mbx   mon_chkr_mbx;
35
36     int esperados = 0;
37     int errores   = 0;
38     int esp_rcbd [drvrs]; // cuantos deberia recibir cada terminal
39
40     ...
41
42     $display("---- resumen ----");
43     $display("paquetes que tomo el DUT (agent_drv)      : %0d", padre_drv.total_enviados());
44     $display("recepciones que junto el monitor        : %0d", padre_mon.total_recibidos());
45     ;
46     $display("recepciones esperadas (suma por terminal) : %0d", total_esperado);
47

```

```

48 // drena lo que el agent_mon junto y lo compara contra lo esperado
49 while (mon_chkr_mbx.num() > 0) begin
50     mon_chkr_mbx.get(m);
51     m.print("prueba_monitor: visto");
52
53     if (m.t_recibido == 0) begin
54         errores++;
55         $display("ERROR: pkg_mon de terminal %0d sin t_recibido", m.terminal);
56     end
57
58     if (m.terminal < 0 || m.terminal >= drvrs) begin
59         errores++;
60         $display("ERROR: pkg_mon con terminal invalida (%0d)", m.terminal);
61     end
62     else if (!buscar_y_sacar(m.terminal, m.destino, m.payload)) begin
63         errores++;
64         $display("ERROR: terminal %0d recibio destino=0x%0h payload=0x%0h sin esperarlo",
65                 m.terminal, m.destino, m.payload);
66     end
67 end
68
69 // lo que quedo sin sacar es lo que se esperaba y nunca llego
70 for (int i = 0; i < drvrs; i++) begin
71     if (esp_destino[i].size() != 0) begin
72         errores++;
73         $display("ERROR: a la terminal %0d le faltaron %0d recepciones",
74                 i, esp_destino[i].size());
75     end
76 end
77
78 if (padre_mon.total_recibidos() != total_esperado) begin
79     errores++;
80     $display("ERROR: el agent_mon junto %0d y se esperaban %0d",
81             padre_mon.total_recibidos(), total_esperado);
82 end
83
84 if (errores == 0) $display("==== PRUEBA MONITOR: PASS ====");
85 else
86     $display("==== PRUEBA MONITOR: FAIL (%0d errores) ====", errores);
87 $finish;
88 end
89
90 initial begin
91     #('PERIODO_CLK * 20000);
92     $display("==== PRUEBA MONITOR: TIMEOUT ====");
93     $finish;
94 end
95 endmodule

```